

PROJECT BRIEF & BUSINESS SCENARIO

API Documentation & Integration Support System

Company: ConnectX Technologies

Department: Software Engineering / Developer Support

Project Type: Multi-Agent AI System

Business Scenario

ConnectX Technologies provides an order-management API that other internal teams can use to retrieve customer and order information.

Developers regularly receive integration requests from teams such as Reporting, Finance, Customer Support, Operations, Logistics, and Management.

Before starting an integration, developers need to understand the business requirement, search the API documentation, identify the correct endpoint, understand required parameters, check authentication requirements, consider pagination and rate limits, and decide how errors should be handled.

At present, developers perform much of this work manually by searching through technical documentation and combining information from different sections.

Your task is to improve this process using a Multi-Agent AI System.

Your Challenge

Build a Multi-Agent System that helps developers analyse an integration requirement and prepare a reliable, source-grounded integration plan using the provided API documentation.

Your system should use the documentation as its main source of truth.

You are responsible for deciding:

- how many agents are needed;
- what responsibility each agent should have;
- how the agents should work together;
- how developer requirements should be interpreted;
- how relevant API information should be found and verified;
- how unsupported requirements should be handled;
- how authentication, pagination, rate limits, and errors should be considered;
- what information should appear in the final integration plan; and
- where developer review is required.

The system should support developers in planning an integration. It should not replace developer validation or automatically make changes to production systems.

BUSINESS RESOURCES & DATA SET

Resource 1: Developer Requirement Dataset

Use the following integration requests as the main input for your system.

R001

Requesting Team	Reporting Team
Business Requirement	Retrieve all customer orders created during a selected date range and save them into the internal reporting system.
Data Needed	Order ID, customer, date, status, total amount
Destination	Internal Reporting System
Frequency	Daily

R002

Requesting Team	Customer Support
Business Requirement	Retrieve the latest information for a specific customer order when a support employee enters the order number.
Data Needed	Order status, items, payment status, delivery status
Destination	Support Dashboard
Frequency	On Demand

R003

Requesting Team	Finance
Business Requirement	Retrieve all completed orders from the previous day for revenue reconciliation.
Data Needed	Order ID, total amount, payment status, completed date
Destination	Finance Sheet
Frequency	Daily

R004

Requesting Team	Operations
Business Requirement	Monitor recently created orders and identify orders that remain pending for too long.
Data Needed	Order ID, created date, status, customer ID
Destination	Operations Tracker
Frequency	Every 2 Hours

R005

Requesting Team	Management
Business Requirement	Prepare a weekly summary of orders by status and total sales value.
Data Needed	Order status, order total, creation date
Destination	Management Report
Frequency	Weekly

R006

Requesting Team	Customer Experience
Business Requirement	Retrieve all orders belonging to a specific customer to help understand their purchase history.
Data Needed	Customer ID, order ID, date, status, total
Destination	Customer Profile
Frequency	On Demand

R007

Requesting Team	Logistics
Business Requirement	Find orders that are ready for shipping so the logistics team can prepare dispatch information.
Data Needed	Order ID, status, delivery address, items
Destination	Logistics Tracker
Frequency	Every Hour

R008

Requesting Team	Audit Team
Business Requirement	Retrieve information about a specific order for an internal audit review.
Data Needed	Complete order details
Destination	Audit Record
Frequency	On Demand

Each request represents a different integration problem. Your system should analyse the requirement before deciding what information from the API documentation is relevant.

Resource 2: ConnectX Orders API Documentation

The following API documentation is fictional and provided only for this project. Your system should treat this documentation as the approved source.

API Overview

API Name	ConnectX Orders API
Base URL	https://api.connectx.demo/v1
Format	JSON

Purpose	The API allows approved applications to retrieve information about customers and orders from the ConnectX order-management platform. The project uses read-only API operations.
---------	---

Endpoint 1: Get a Single Order

Method	GET
Endpoint	/orders/{order_id}
Purpose	Retrieves detailed information about one specific order.

Required Path Parameter

Parameter	Required	Description	Example
order_id	Yes	Unique ID of the order	ORD-10452

Example Request

GET /orders/ORD-10452

Example Response

```
{
  "order_id": "ORD-10452",
  "customer_id": "CUS-3021",
  "created_at": "2026-09-03T10:35:00Z",
  "status": "completed",
  "payment_status": "paid",
  "total": 12500,
  "currency": "NPR",
  "items": [
    {
      "product_id": "PRD-201",
      "name": "Wireless Keyboard",
      "quantity": 2,
      "unit_price": 6250
    }
  ]
}
```

Endpoint 2: List Orders

Method	GET
Endpoint	/orders
Purpose	Retrieves a list of orders. The endpoint supports filters that allow developers to retrieve selected records.

Available Query Parameters

Parameter	Required	Description	Example
start_date	No	Return orders created on or after this date	2026-09-01
end_date	No	Return orders created on or before this date	2026-09-05
status	No	Filter orders by status	completed

Parameter	Required	Description	Example
customer_id	No	Return orders belonging to a specific customer	CUS-3021
page	No	Page number for paginated results	1
limit	No	Number of results returned per page	50

Supported Order Status Values

Status	Meaning
pending	Order has been created but has not entered processing
processing	Order is currently being prepared
ready_for_shipping	Order is ready to be dispatched
shipped	Order has been dispatched
completed	Order has been completed
cancelled	Order has been cancelled

Example Request

```
GET /orders?start_date=2026-09-01&end_date=2026-09-05&status=completed
```

Example Response

```
{
  "page": 1,
  "limit": 50,
  "total_records": 2,
  "orders": [
    {
      "order_id": "ORD-10452",
      "customer_id": "CUS-3021",
      "created_at": "2026-09-03T10:35:00Z",
      "status": "completed",
      "payment_status": "paid",
      "total": 12500,
      "currency": "NPR"
    },
    {
      "order_id": "ORD-10453",
      "customer_id": "CUS-3040",
      "created_at": "2026-09-04T09:10:00Z",
      "status": "completed",
      "payment_status": "paid",
      "total": 8400,
      "currency": "NPR"
    }
  ]
}
```

Endpoint 3: Get Customer Details

Method	GET
Endpoint	/customers/{customer_id}
Purpose	Retrieves basic information about a specific customer.

Required Path Parameter

Parameter	Required	Description	Example
customer_id	Yes	Unique customer identifier	CUS-3021

Example Response

```
{
  "customer_id": "CUS-3021",
  "name": "Aarav Sharma",
  "email": "aarav@example.demo",
  "customer_since": "2024-05-14"
}
```

Endpoint 4: Get Order Delivery Information

Method	GET
Endpoint	/orders/{order_id}/delivery
Purpose	Retrieves delivery information for a specific order.

Example Response

```
{
  "order_id": "ORD-10452",
  "delivery_status": "shipped",
  "carrier": "Demo Express",
  "tracking_reference": "DX-558402",
  "estimated_delivery": "2026-09-06"
}
```

Resource 3: Authentication and Usage Rules

Authentication Rules

The ConnectX Orders API uses API key authentication.

The API key must be included in the request header.

Header Name	X-API-Key
-------------	-----------

Example

```
X-API-Key: YOUR_API_KEY
```

API credentials should be treated as sensitive information. They must not be exposed in prompts, spreadsheets, workflow outputs, screenshots, reports, or presentations.

For this classroom project, students should use fictional credentials or mocked responses.

API Access Rules

Area	Rule
API Access	Only approved applications may access the API
Project Access	Use fictional credentials or mocked API responses
Available Operations	Only read-only GET operations are included in this project
Customer Data	Retrieve only the customer information required for the stated business purpose
API Keys	Credentials must not be exposed in project outputs
Production Changes	The project must not create, update, or delete real orders
Final Validation	A developer must review an integration plan before implementation

Rate Limit

The API allows a maximum of 100 requests per minute.

If the limit is exceeded, the API returns a rate-limit error.

Your solution should consider how this limitation may affect different integration requirements.

Pagination

The List Orders endpoint returns a maximum of 50 orders per request.

If more records exist, additional pages must be requested.

Example

GET /orders?page=1&limit=50

followed by:

GET /orders?page=2&limit=50

Your system should recognize situations where pagination may be relevant.

Resource 4: API Error Reference

Use the following error information when planning integrations.

HTTP Status	Meaning	Guidance
200	Request successful	Continue processing the returned information
400	Invalid request	Check the request structure and parameters
401	Authentication failed	Check the API credentials
403	Access denied	Check whether the application has permission to access the resource
404	Resource not found	Check the requested order, customer, or endpoint
429	Too many requests	Reduce request frequency and retry after an appropriate delay
500	Server error	Record the error and retry later or send for technical review
503	Service unavailable	Pause processing and retry later

Error Handling Rules

Your solution should consider how unsuccessful API requests are handled.

- Repeated failed requests should not continue indefinitely.
- Authentication failures should require developer review.
- Rate-limit errors should not result in immediate repeated requests.
- Missing records should not cause the complete integration process to fail unnecessarily.
- Server-side errors should be recorded for investigation.

Resource 5: Integration Review Rules

Your system should use the following rules when preparing and reviewing an integration plan.

Endpoint Accuracy

Any endpoint recommended by the system must exist in the provided documentation. The system must not invent endpoints.

Parameter Accuracy

Required parameters must be included. Optional parameters should only be used when they support the business requirement.

Authentication

The integration plan should recognize authentication requirements without exposing credentials.

Pagination

Pagination should be considered when a request may return more records than the API allows in a single response.

Error Handling

The proposed integration should consider relevant API errors and how the workflow should respond.

Data Minimization

The integration should retrieve and store only the information required for the stated business purpose.

Unsupported Requirements

If the API documentation does not support a requirement, your system should clearly identify the limitation. It must not invent functionality simply to satisfy the request.

Human Review

The AI system may prepare an integration plan, but a developer must review and validate the plan before implementation.

ASSESSMENT REQUIREMENTS & SUBMISSION

Multi-Agent Requirement

- This project must be completed as a Multi-Agent System.
- Your agents should have different and meaningful responsibilities.
- You are responsible for deciding how the integration-planning problem should be divided between agents.
- You should be able to explain:
 - why each agent is required;
 - what information each agent receives;
 - what each agent produces;
 - how agents communicate or hand work to one another; and
 - how your design reduces incorrect or unsupported API recommendations.
- Simply passing the same developer requirement through several AI nodes will not be considered a strong Multi-Agent design.

What Your System Must Demonstrate

Your completed system should demonstrate that it can:

- understand different integration requirements;
- use the provided API documentation as its source of truth;
- identify relevant API capabilities;
- use endpoints and parameters correctly;
- recognize authentication requirements;
- consider pagination where appropriate;
- consider API rate limits;
- plan for common API errors;
- avoid retrieving unnecessary customer information;
- identify unsupported requirements;
- avoid inventing API functionality;
- prepare a useful integration plan; and
- keep developers responsible for final validation and implementation.

Important Rules

- Your system should remain grounded in the provided API documentation.
- Do not invent endpoints.
- Do not invent parameters.
- Do not invent response fields.

- Do not assume an API capability exists unless it is supported by the provided documentation.
- Do not expose credentials.
- Do not use real customer information.
- Do not perform real production changes.
- If documentation is incomplete or does not support a requirement, clearly state that further developer investigation is needed.

Testing Requirement

You are responsible for designing the testing approach for your system.

Your testing should demonstrate that the Multi-Agent System can handle different types of developer requirements rather than producing the same integration plan every time.

Your testing should provide enough variation to demonstrate how the system handles:

- different API requirements;
- single-record and multiple-record requests;
- different filters and parameters;
- large result sets;
- authentication requirements;
- API errors;
- missing information;
- unsupported requirements; and
- requirements that may need more than one API endpoint.

Keep evidence of your testing and any improvements you make.

Testing Record

You may use this format or create your own suitable testing record.

Test	What You Tested	Expected Result	Actual Result	Issue Found	Improvement Made
1					
2					
3					
4					

Final Project Submission

Your final submission should show your complete analysis, design, and problem-solving process.

1. Problem Understanding

Explain the API integration-support problem faced by ConnectX Technologies.

2. Existing Process

Explain how developers would normally analyse requirements and search API documentation without your system.

3. AI Opportunity

Explain where AI can support the integration-planning process and where normal workflow logic or developer judgment is more suitable.

4. Solution Design

Explain the system you designed and why you chose that approach.

5. Multi-Agent Design

Show the agents you created, their responsibilities, and how they work together.

6. Workflow

Show how a developer requirement moves through your system until an integration plan is produced.

7. Working System

Demonstrate the actual Multi-Agent System.

8. Documentation Use

Show how your system uses the API documentation and avoids unsupported assumptions.

9. Testing and Improvement

Show how you tested the system, what problems you found, and what you improved.

10. Final Integration Plan

Show at least one complete integration plan generated by your system.

11. Business and Technical Value

Explain how your solution could reduce the time developers spend searching documentation or preparing initial integration plans.

12. Limitations

Explain what the AI cannot verify and where incorrect interpretation could occur.

13. Human Oversight

Explain what developers must still review, test, and approve before implementation.

Final Presentation

Your presentation should clearly communicate the following.

The Problem

What developer-support problem are you trying to solve?

Your Approach

How did you analyse the problem and decide what your Multi-Agent System needed to do?

Your Multi-Agent System

What agents did you design and why?

Live Demo

Provide a developer requirement and demonstrate how your system turns it into an integration plan.

Show how the system uses the provided documentation rather than relying on unsupported assumptions.

Testing

Demonstrate how the system behaves when the requirement, available documentation, or API conditions change.

Impact and Limitations

Explain how the system could support developers, where mistakes may still occur, and why developer validation remains necessary.

Your Goal

The goal is not simply to ask AI which API endpoint to use.

Your goal is to design a Multi-Agent System that can reason about an integration requirement, work from technical documentation, identify limitations, prepare a useful plan, and recognize when human technical judgment is required.

You are expected to analyse the problem, design the agents, decide how information should move through the system, select suitable tools, test your design, and improve it based on what you discover.

A strong project should demonstrate technical reasoning, source-grounded AI use, careful handling of API limitations, and responsible human oversight.